

Chapter – 6

Servlets and JSP

HTTP

The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers. HTTP works as a request – response protocol between client and server.

Example: A client send an HTTP request to the server, then the server returns a response to the client. The response contains status information about the request and may also contain the requested content

HTTP methods

- **GET**

GET is used to request data from a specified resource.

- **POST**

POST is used to send data to a server to create / update a resource.

- **PUT**

PUT is used to send data to a server to create / update a resource.

The difference between POST and PUT is that PUT request are idempotent i.e calling the same PUT request multiple times will always produce the same result. In contrast, calling a POST request repeatedly have side effect of creating the same resource multiple times.

- **HEAD**

Head is almost identical to GET, but with the response body.

i.e if GET /users returns a list of a users, then HEAD / users will make the same request but will not return the list of users. A HEAD request is useful for checking what a GET request will return before actually making a GET request.

- **DELETE**

The DELETE method deletes the specified resource.

- **PATCH**

The PATCH method is used to apply partial modification to a resource.

- **OPTIONS**

The OPTIONS method describe the communication options for the target resource.

- **CONNECT**

The CONNECT method is used to start a two – way communication (a tunnel) with the requested resources

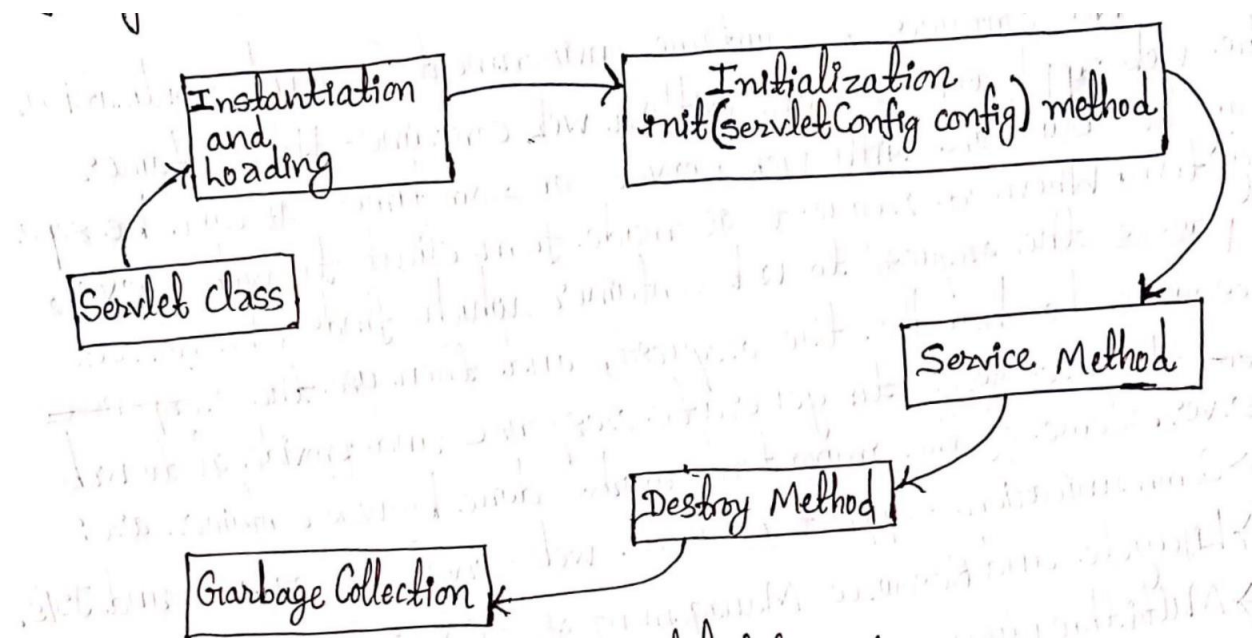
- **TRACE**

The TRACE method is used to perform a message loop – back test that tests the path for the target resource.

Introduction to Servlets

- Servlets are small programs that execute on the server side of a web connection.
- Just as applets dynamically extend the functionality of a web browser servlets dynamically extend the functionality of a Web server.
- It is a java programming language class used to extend the capabilities of servers that host applications accessed via a request – response programming model.
- Although servlets can respond to any type of request they are commonly used to extend the applications hosted by web servers.
- For such applications, java servlet technology define HTTP – Specific servlet classes.
- The javax.servlet and javax.servlet.http package provide interfaces and classes for writing servlets .
- All servlets must implement the servlet interface, which defines life – cycle methods.

Life cycle of a servlets



The servlet life cycle had 5 stages in general. Among them three methods are central to the life cycle of a servlet. These are init(), service() and destroy().

1. Loaded and Instantiated

The class loader is responsible to load the servlet class. Then it creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

2. Initialized by calling the init() method

The init method is used to initialize the servlet.

Syntax of the init() method is as follows

Public void init(ServletConfig config) throws ServletException

3. Process a clients request by invoking service() method

The web container calls the service method each time when request for the servlet is received. The syntax is as follows:

Public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException

4. Terminated by calling the destroy() method

It gives the servlet an opportunity to clean up any resource for example memory, thread etc

Syntax

Public void destroy()

5. Garbage collected by the JVM

Once the servlet is destroyed, garbage collection component of JVM is responsible collecting the garbage's.

Servlet APIs

- Two packages contain the classes and interfaces that are required to build servlets. These are javax.servlet and javax.servlet.http .
- They constitute the servlet API.
- These packages are not part of the java core packages.
- Instead, they are standard extensions. Therefore, they are not included in the Java software Development kit.
- We must download Tomcat or Glass Fish server to obtain their functionality.

The javax.servlet package

This package contains a number of interfaces and classes that establish the framework in which servlets operate. Following are some of the interfaces and classes.

Interface	Description
Servlet	Declares life cycle methods for a servlet.
ServletConfig	Allows servlets to get initialization parameters.
ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.

Class	Description
GenericServlet	Implements the Servlet and ServletConfig interfaces.
ServletInputStream	Provides an input stream for reading requests.
ServletOutputStream	Provides an output stream for writing response.
ServletException	Indicates a servlet error occurred.

The javax.servlet.http package

This package contains a number of interfaces and classes that are commonly used by servlet developers. Following are some of the interfaces and classes.

Interface	Description
HttpServletRequest	Enables servlets to read data from HTTP request.
HttpServletResponse	Enables servlets to write data to an HTTP response.
HttpSession	Allows session data to be read and write.

Class	Description
Cookie	Allow state information to be stored on client machine.
HttpServlet	Provides methods to handle HTTP requests and responses.
HttpSessionEvent	Encapsulates a session – changed event

Writing Servlet Programs

The servlet program can be created by three ways:

- I. By implementing servlet interface
- II. By inheriting GenericServlet class
- III. By inheriting HttpServlet class

The mostly used approach is by extending HttpServlet because it provides http request specific method such as doGet(), doPost(), doHead() etc.

There are six steps to write Servlet Program (Here we are using **apache tomcat server**)

- I. Create a directory structure
- II. Create a Servlet
- III. Compile the Servlet
- IV. Create a deployment descriptor
- V. Start the server and deploy the project
- VI. Access the servlet

Follow this video tutorial link to integrate Tomcat Server with Netbeans
<https://www.youtube.com/watch?v=a6dOGqC8XLA>

1. Servlet Interface

Servlet interface provides common behavior to all the servlets. Servlet interface needs to be implemented for creating any servlet (either directly or indirectly). It provides three life cycle methods that are used to initialize the servlet, to service the requests and to destroy the servlet and two non life cycle methods that are used to get servlet information and servlet configuration.

Example

```
import java.io.*;
import jakarta.servlet.ServletException;
import jakarta.servlet.*;

public class First implements Servlet
{
    private ServletConfig config = null;

    @Override
    public void init(ServletConfig config) throws ServletException
    {
        this.config = config;
        System.out.println("Servlet is initialized");
    }

    @Override
    public void service(ServletRequest request, ServletResponse response) throws IOException, ServletException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<b> Hi we are student of Everest Engineering college </b>");
        out.println("</body></html>");
    }

    @Override
    public void destroy()
    {
        System.out.println("Servlet is destroyed");
    }

    @Override
    public ServletConfig getServletConfig()
    {
        return config;
    }

    @Override
    public String getServletInfo()
    {
        return "Implementing Servlet Interface";
    }
}
```

Output

Hi we are student of Everest Engineering college

2. GenericServlet class

GenericServlet class implements Servlet, ServletConfig and Serializable interfaces. It provides the implementation of all the methods of these interfaces except the service method. GenericServlet class can handle any type of request so it is protocol – independent. We can create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

Example

```
import java.io.*;
import jakarta.servlet.*;

public class DemoGeneServlet extends GenericServlet
{
    @Override
    public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<head><title>GenericServlet Example</title></head>");
        out.println("<body>");
        out.println("<h2>Hello from GenericServlet!</h2>");
        out.println("<p>This servlet is created using the GenericServlet class.</p>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Output

Hello from GenericServlet!

This servlet is created using the GenericServlet class.

3. HttpServlet class

The HttpServlet class extends the GenericServlet class and implements Serializable interface. It provides http specific methods such as doGet, doPost etc. doGet() method handles the GET request and doPost() handles the POST request. Get request is the default request.

Example

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class DemoHttpServlet extends HttpServlet
{
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException, ServletException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<b> Hi, students How are you? </b>");
        out.println("</body></html>");
    }
}
```

Output

Hi, students How are you?

Reading and Processing Forms

The **servletRequest** interface includes method that allow you to read the names and values of parameter that are included in a client request. We will develop a servlet that illustrates their use. The example contains two files. A web page is defined in **Register1.html** and a servlet is defined in **RegisterServlet1.java**.

The HTML source code for **Register1.html** is shown below. It defines a table that contains two labels and two text fields. One of the labels is Employee and the other is phone. There is also a submit button.

Register1.html

```
<html>
<body>
  <center>
    <form name="Form1" method="post" action="RegisterServlet1">
      <table>
        <tr>
          <td><b>Employee:</b></td>
          <td><input type="text" name="employee" size="25"></td>
        </tr>
        <tr>
          <td><b>Phone:</b></td>
          <td><input type="text" name="phone" size="25"></td>
        </tr>
      </table>
      <input type="submit" value="Submit">
    </form>
  </center>
</body>
</html>
```

The source code for **RegisterServlet1.java** is shown below. The **service()** method is overridden to process client requests. The **getParameterNames()** method returns an enumeration of the parameter names. There are processed in a loop. You can see that the parameter name and value are output to the client. The parameter value is obtained via the **getParameter()** method.

RegisterServlet1.java

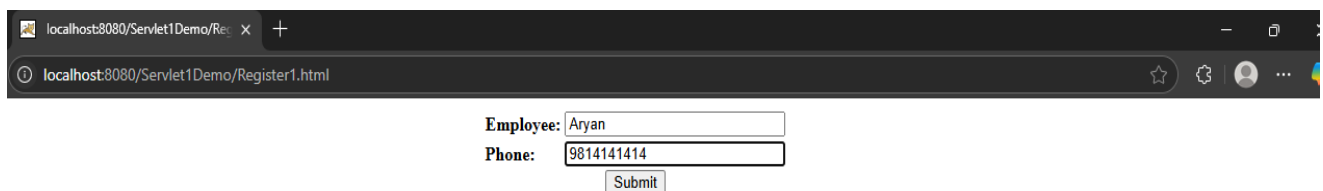
```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class RegisterServlet1 extends HttpServlet
{
    @Override
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

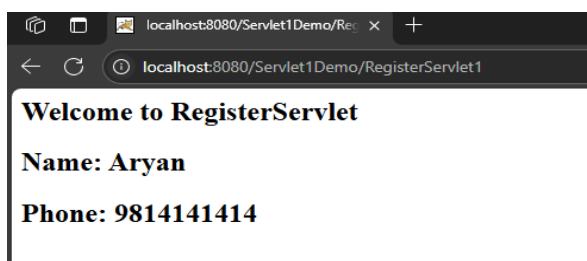
        String name = request.getParameter("employee");
        String phone = request.getParameter("phone");

        out.println("<h2>Welcome to RegisterServlet</h2>");
        out.println("<h2>Name: " + name + "</h2>");
        out.println("<h2>Phone: " + phone + "</h2>");
    }
}
```

Output



A screenshot of a web browser window. The address bar shows 'localhost:8080/Servlet1Demo/Register1.html'. The page content includes two input fields: 'Employee: Aryan' and 'Phone: 9814141414'. Below these fields is a 'Submit' button.



A screenshot of a web browser window showing the output of the servlet. The address bar shows 'localhost:8080/Servlet1Demo/RegisterServlet1'. The page content displays the following text:

Welcome to RegisterServlet

Name: Aryan

Phone: 9814141414

Handling HTTP Request

The `HttpServlet` class provides specialized methods that handle the various types of HTTP requests. A servlet developer typically overrides one of these methods. These methods are **`doDelete()`**, **`doGet()`**, **`doHead()`**, **`doOption()`**, **`doPost()`** and **`doTrace()`**. However, the GET and POST requests are commonly used when handling forms input.

Handling HTTP Get Requests

Here we develop a servlet that handles an HTTP Get request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in **`ColorGet2.html`** and a servlet is defined in **`ColorGetServlet2.java`**. The source code for **`ColorGet2.html`** is shown below. It defines a form that contain a select element and a submit button.

ColorGet2.html

```
<html>
<body>
  <center>
    <form name="Form1" method="get" action="ColorGetServlet2">
      <b>Color:</b>
      <select name="color" size="1">
        <option value="Red">Red</option>
        <option value="Green">Green</option>
        <option value="Blue">Blue</option>
      </select>
      <input type="submit" value="Submit">
    </form>
  </center>
</body>
</html>
```

the source code for **`ColorGetServlet2.java`** is shown below. The `doGet()` method is overridden to process any HTTP GET request that are sent to this servlet. It uses `getParameter()` method of `HttpServletRequest` to obtain the selection that was made by the user.

ColorGetServlet2.java

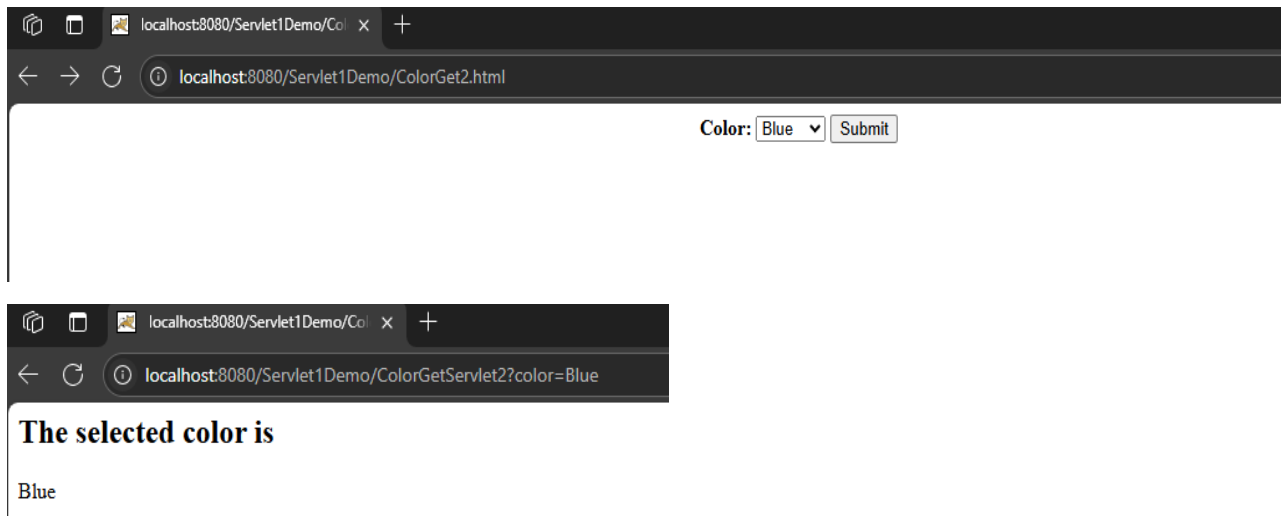
```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ColorGetServlet2 extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String color = request.getParameter("color");

        out.println("<h2>The selected color is</h2>");
        out.println(color);
        out.close();
    }
}
```

Output



Handling HTTP POST Request

Here we will develop a servlet that handles an **HTTP POST** request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in **ColorPost2.html** and a servlet is defined in **ColorPostServlet2.java**.

The HTML source code for **ColorPost2.html** is shown below. It is identical to **ColorGet2.html** except that the method parameter for the form tag explicitly specifies that the **POST** method should be used and the action parameter for the form tag specifies a different servlet.

Colorpost2.html

```
<html>
<body>
  <center>
    <form name="Form1" method="post" action="ColorPostServlet2">
      <b>Color:</b>
      <select name="color" size="1">
        <option value="Red">Red</option>
        <option value="Green">Green</option>
        <option value="Blue">Blue</option>
      </select>
      <input type="submit" value="Submit">
    </form>
  </center>
</body>
</html>
```

The source code for **ColorPostServlet2.java** is shown below. The **doPost()** method is overridden to process any HTTP POST request that are sent to this servlet. It uses the **getParameter()** method of **HttpServletRequest** to obtain the selection that was made by the user. A response is then formulated.

ColorPostServlet2.java

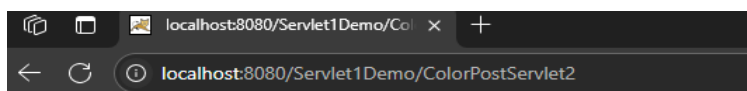
```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ColorPostServlet2 extends HttpServlet
{
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String color = request.getParameter("color");

        out.println("<h2>The selected color is</h2>");
        out.println(color);
        out.close();
    }
}
```

Output



The selected color is

Green

Database access with servlets

Database access with servlets is same as JDBC , that we have discussed earlier. Only the difference is that we have to put mysql-connector.jar file in libraries directory.

Insert1.html

```
<html>
<body>
  <center>
    <form name="Form1" method="get" action="InsertServlet1">
      <table>
        <tr>
          <td><b>Roll_no</b></td>
          <td><input type="text" name="roll_no" size="25"></td>
        </tr>
        <tr>
          <td><b>Name</b></td>
          <td><input type="text" name="name" size="25"></td>
        </tr>
        <tr>
          <td><b>Email</b></td>
          <td><input type="text" name="email" size="25"></td>
        </tr>
        <tr>
          <td><b>Address</b></td>
          <td><input type="text" name="address" size="25"></td>
        </tr>
      </table>
      <input type="submit" value="Submit">
      <input type="submit" value="reset">
    </form>
  </center>
</body>
</html>
```

InsertServlet1.java

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class InsertServlet1 extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String Roll_no = request.getParameter("roll_no");
        String Name = request.getParameter("name");
        String Email = request.getParameter("email");
        String Address = request.getParameter("address");

        try
        {
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");

            Statement statement = conn.createStatement();
            statement.executeUpdate("insert into nesfield_student values('" + Roll_no + "','"
                + "''" + Name + "','" + Email + "','" + Address + "')");
            out.println("<h1>Your data is successfully submitted</h1>");
        }
        catch (Exception e)
        {
            System.out.println("Some error: " + e);
        }
    }
}
```

Output

← → ↻ localhost:8080/Servlet1Demo/Insert1.html

Roll_no	7
Name	Cristiano Ronaldo
Email	Ronaldo@gmail.com
Address	Portugal

← ↻ localhost:8080/Servlet1Demo/InsertServlet1?roll_no=7&name=Cristiano+Ronaldo&email=Ronaldo%40gmail.com&address=Portugal

7

Your data is successfully submitted

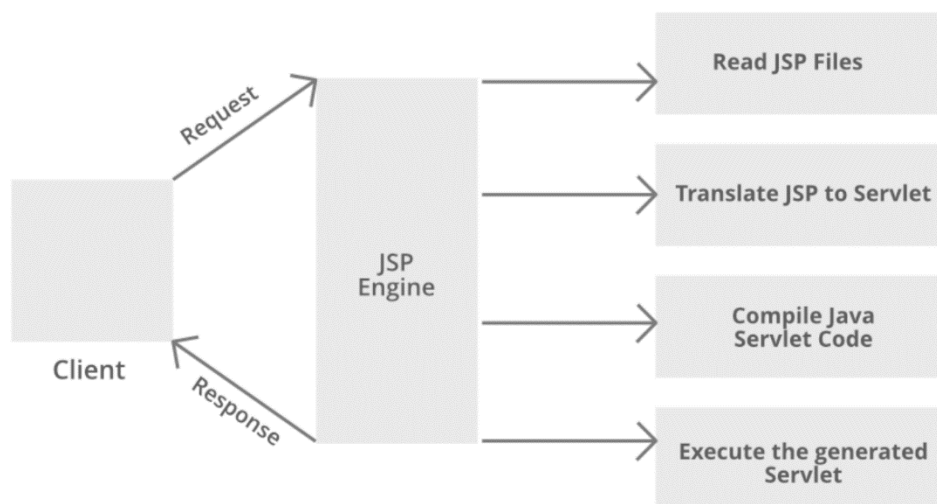
Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Bhaktapur
2	Hari	Hari@gmail.com	Pokhara
3	Zenith	Shyam@gmail.com	Dhangadi
5	Harryporter	Harry@gmail.com	Baglung
4	Eliza	Eliza@gmail.com	Lalitpur
7	Cristiano Ronaldo	Ronaldo@gmail.com	Portugal

JSP(Java Server Page)

Java Server Page (JSP) is a server side programming technology that enables the creation of dynamic, platform – independent method for building web based applications. JSP have access to the entire family of java APIs, including the JDBC API to access enterprise databases. It can be thought of as an extension to servlet because it provides more functionality than servlet. A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than servlet because we can separate designing and development.

Web developers write JSPs as text files that combine HTML or XHTML code, XML elements and embedded JSP actions and commands.

JSP Process



- The client navigates to a file ending with the **.jsp extension** and the browser initiates an HTTP request to the webserver. For example, the user enters the login details and submits the button. The browser requests a status.jsp page from the webserver.
- If the compiled version of JSP exists in the web server, it returns the file. Otherwise, the request is forwarded to the JSP Engine. This is done by recognizing the URL ending with **.jsp** extension.
- The JSP Engine loads the JSP file and translates the JSP to Servlet(Java code). This is done by converting all the template text into `println()` statements and JSP elements to Java code. This process is called **translation**.
- The JSP engine compiles the Servlet to an executable **.class** file. It is forwarded to the Servlet engine. This process is called **compilation** or **request processing phase**.
- The **.class** file is executed by the Servlet engine which is a part of the Web Server. The output is an HTML file. The Servlet engine passes the output as an HTTP response to the webserver.
- The web server forwards the HTML file to the client's browser.

JSP Access Model

In JSP, the access model refers to the way in which the built – in objects and other resources are made available to the JSP page. There are three main access models in JSP, which are used according to the need of application.

i. Page – Base Access

In this model, the built-in objects are made available to the JSP page through local variables.

Syntax: request.getParameter(“ParaName”)

ii. Action – Based Access

In this mode, the built – in objects are made available to the JSP through the use of custom tags.

iii. Bean – Based Access

In this model, the built – in objects are made available to the JSP page through JavaBeans.

Syntax: beanName.propertyName

JSP Syntax

There are five main tags that all together form JSP syntax. These tags are Declaration tags, Expression tags, Directive tags, Scriptlet tags, and Action tags.

Declaration tag: This tag is used to declare variable and methods

Syntax: < % ! Variable or method declaration %>

Example

```
< % !  
    private int counter = 0;  
    private String getAccount(int accountNO);  
% >
```

Expression tag: It is used to insert a single java expression directly into the response message by placing inside a out.print() method.

Syntax: < % = Java Expression % >

Example

Date : < % = new java.util.Date() % >

Directive tags: The directive tags gives special information about the page to the JSP Engine.

Syntax: `<%@ directive attribute = "value" %>`

Example:

```
< % @ page language = "java" % >  
< % @page import = "java.util.* " % >
```

Scriptlet tag: This tag is used to execute java code in JSP.

Syntax: `< % java code % >`

Example:

```
< %  
    String username = "Nishanta";  
    out.println(username);  
% >
```

Action tag: The action tag are used to control the flow between pages and to use java beans. Some of the action tags supported by JSP are : jsp:forward, jsp:include, jsp:param.

Syntax: `<jsp : forward page = "printdate.jsp" />`

JSP comments: The JSP comment is used when we want to hide or “comment out” part of the JSP page.

Syntax: `<% -- comment -- %>`

Example:

```
< % - - This is JSP Comment - - % >
```

JSP Implicit Objects

In JSP, implicit objects are objects that are automatically available to the JSP page and do not need to be explicitly created. Some of the implicit objects in JSP are as follows:

Request object: It represents the HTTP request received by the web server and provides access to request – specific information such as parameters, headers and session attributes.

Response object: It represents the HTTP response that will be sent back to the client and allows the JSP to control the content and headers of the response.

Page context object: Provides access to various components of the JSP environment, such as the ServletContext, HttpSession and JspWriters.

Session object: Represents the HTTP session for a user and allows the JSP to access and modify session – level data.

Page object: Represents the current JSP page and provides access to the pages attributes and methods.

These implicit objects can be accessed using standard java syntax within a JSP page. For example, to access the request object, we would use ‘request’ and to access a parameter passed in the request we would use ‘*request.getParameter(“paramName”)*’.

Object Scope

In JSP, object scope refers to the availability of an object within the page, application or session. The scope of an object determines where it can be accessed and for how long it is available. There are four object scopes in JSP:

Page Scope: an object with page scope is available only within the current JSP page and is not accessible from other pages.

```
<jsp:useBean id = “employee” class=“EmployeeBean” scope = “page” />
```

Request Scope: An object with request scope is available within the current request and response cycle and is not accessible from other requests

```
<jsp:useBean id = “employee” class=“EmployeeBean” scope = “request” />
```

Session Scope: An object with session scope is available to all pages within a user’s session and is not accessible from other sessions.

```
<jsp:useBean id = “employee” class=“EmployeeBean” scope = “session” />
```

Application Scope: An object with application scope is available to all pages in the web application and is not accessible from other applications.

```
<jsp:useBean id = “employee” class=“EmployeeBean” scope = “application” />
```

Simple JSP program

```
<html>
  <head>
    <title>Simple JSP Page</title>
  </head>
  <body>
    <h1>Displaying Pokhara University 10 times</h1>
    <table>
      <%
        for( int i = 1; i<=10; i++)
        {
          %>
            <tr>
              <td>Pokhara University </td>
            </tr>
          <%
        }
      %>
    </table>
  </body>
</html>
```

Displaying Pokhara University 10 times

[illegible]

Form Processing in JSP

Form processing in JSP is similar to form processing in servlets. We can use request object to read parameter values from HTML file to another or another JSP file. Methods supported by request object are similar to methods used in servlets.

processFile.html

```
<html>
<body>
  <center>
    <form name="Form1" method="post" action="ProcessFile.jsp">
      <table>
        <tr>
          <td><b>Employee:</b></td>
          <td><input type="text" name="employee" size="25"></td>
        </tr>
        <tr>
          <td><b>Phone:</b></td>
          <td><input type="text" name="phone" size="25"></td>
        </tr>
      </table>
      <input type="submit" value="Submit">
    </form>
  </center>
</body>
</html>
```

ProcessFile.jsp

```
<html>
<body>
  <%
    String Name = request.getParameter("employee");
    String Phone = request.getParameter("phone");
  %>

  <center>
    <h2> Hello, <%= Name + " your phone number is " + Phone %> </h2>
  </center>
</body>
</html>
```

Output



Hello , Sarada your phone number is 9861242008

Database Access with JSP

We also access database by using JSP. All JDBC APIs can be accessed from JSP. We have put all codes related to database access within the expression tag,

Insert1.html

```
<html>
<body>
  <center>
    <form name="Form1" method="post" action="DatabaseJSP1.jsp">
      <table>
        <tr>
          <td><b>Roll_no</b></td>
          <td><input type="text" name="roll_no" size="25"></td>
        </tr>
        <tr>
          <td><b>Name</b></td>
          <td><input type="text" name="name" size="25"></td>
        </tr>
        <tr>
          <td><b>Email</b></td>
          <td><input type="text" name="email" size="25"></td>
        </tr>
        <tr>
          <td><b>Address</b></td>
          <td><input type="text" name="address" size="25"></td>
        </tr>
      </table>
      <input type="submit" value="Submit">
      <input type="submit" value="reset">
    </form>
  </center>
</body>
</html>
```

DatabaseJSP.jsp

```
<html>
<body>
    <%
        String Roll_no = request.getParameter("roll_no");
        String Name = request.getParameter("name");
        String Email = request.getParameter("email");
        String Address = request.getParameter("address");
    %>
    <center>
        <%@page import = "java.sql.*" %>
        <%
            String driver = "com.mysql.cj.jdbc.Driver";
            String databaseUrl = "jdbc:mysql://localhost:3306/nesfield";
            String username = "root";
            String password = "";
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(databaseUrl, username, password);
            System.out.println("Database connected");

            Statement statement = conn.createStatement();
            statement.executeUpdate("insert into nesfield_student values('" + Roll_no + "', "
                + "'" + Name + "', '" + Email + "', '" + Address + "')");
            out.println("<h1>Your data is successfully submitted</h1>");
        %>
    </center>
</body>
</html>
```

Output

localhost:8080/Servlet1Demo/Insert1.html

Roll_no
Name
Email
Address

localhost:8080/Servlet1Demo/DatabaseJSP1.jsp

Your data is successfully submitted

Roll_no	Name	Email	Address
1	Nishanta	Nishanta@gmail.com	Bhaktapur
2	Hari	Hari@gmail.com	Pokhara
3	Zenith	Shyam@gmail.com	Dhangadi
5	Harryporter	Harry@gmail.com	Baglung
4	Eliza	Eliza@gmail.com	Lalitpur
7	Cristiano Ronaldo	Ronaldo@gmail.com	Portugal
13	Krishna Poudel	youaresuddenly@gmail.com	Bihar, Patna
10	Messi	Messi@gmail.com	Argentina

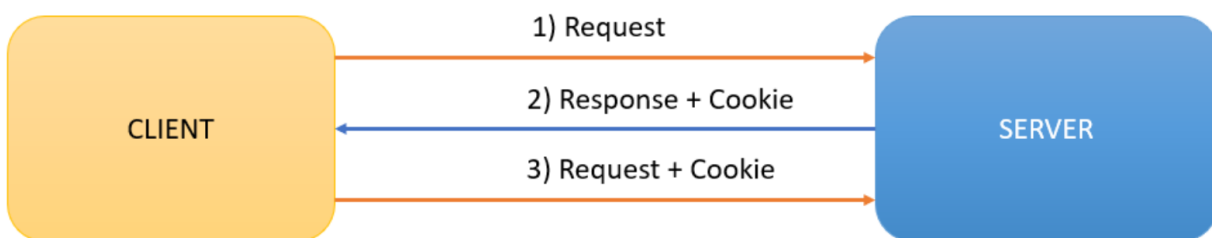
Difference between Servlet and JSP

Servlets	JSP
Servlet is a pure java code.	JSP is a tag based approach.
We write HTML in Servlet code.	We write JSP code in HTML.
Servlet is faster than JSP.	JSP is slower than Servlet.
Writing code for servlet is harder.	Writing code for JSP is easier.
Servlet can accept all protocol request.	JSP only accept http request.
Modification in Servlet is a time consuming task because it includes reloading.	JSP modification is fast, we just need to click refresh button.
Servlet do not have implicit objects.	JSP have implicit objects.
In MVC pattern, servlet act as a controller.	In MVC pattern, JSP act as a view.

Handling cookie in Servlet

A cookie is a small piece of information that is persisted between the multiple client request. A cookie has a name, a single value and optional attributes such as comment, path and domain qualifiers, a maximum age and a version number. There are three steps involved in identifying returning user:

- Server script sends a set of cookies to the browser. For example name, age or identification number etc.
- Browser stores this information on local machine for future use.
- When next time browser sends any request to web server then it sends those cookies information to the server and server use that information to identify the user.



Advantage of cookie

- Simplest techniques of maintaining the state.
- Cookies are maintained at a client side.

Disadvantage of cookie

- It will not work if cookie is disabled from the browser.
- Only textual information can be set in cookie object.

Constructor of cookie class

Constructor	Description
Cookie()	Construct a cookie.
Cookie(String name, string val)	Construct a cookie with a specified name and value.

Useful methods of cookie class

Method	Description
setMaxAge(int expiry)	Sets the maximum age of the cookie in seconds
String getName()	Returns the name of the cookie
String getValue()	Returns the value of the cookie
Void setName(String name)	Change the name of the cookie.
Void setValue(String value)	Change the value of the cookie

Methods required for using cookie

- **Public void addCookie(Cookie ck)**

Method of HttpServletResponse interface is used to add cookie in response object.

- **Public cookie[] getCookies()**

Method of HttpServletRequest interface is used to return all the cookie from the browser.

For example

The HTML source code for **Cookie.html** is show below. This page contains text fields in which a value can be entered. There is also a submit button on the page. When this button is pressed, the value in the text field is sent to **SetServlet.java**

Cookie.html

```
<html>
<body>
<center>
<form name="Form1" method="post" action="SetServlet">
  Name: <input type="text" name="unname">
  <input type="submit" value="Submit">
</form>
</center>
</body>
</html>
```

The source code for **SetServlet.java** is shown below. It gets the value of a parameter named “**unname**”. Then it creates a Cookie object that has the name “**name**” and contains the value of the “**unname**” parameter. The cookie is the added to the header of HTTP response via the **addCookie()** method. A feedback message is then written to the browser.

SetServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class SetServlet extends HttpServlet
{
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("uname");
        out.println("Welcome " + name);

        // Creating submit button
        out.println("<form action='ReadServlet' method='post'>");
        out.println("<input type='submit' value='go'>");
        out.println("</form>");
        out.println("</body>");
        out.println("</html>");

        // Create cookie
        Cookie ck = new Cookie("name", name);
        // Add cookie to HTTP response
        response.addCookie(ck);

        out.close();
    }
}
```

The source code for **ReadServlet.java** is shown below. It involves the **getCookies()** method to read any cookies that are included in the HTTP Get request. The names and value of these cookies are then written to the HTTP response.

ReadServlet.java

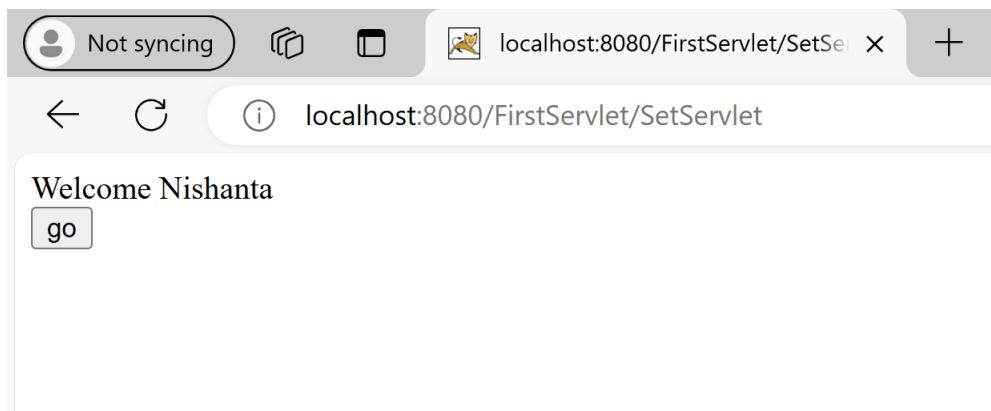
```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

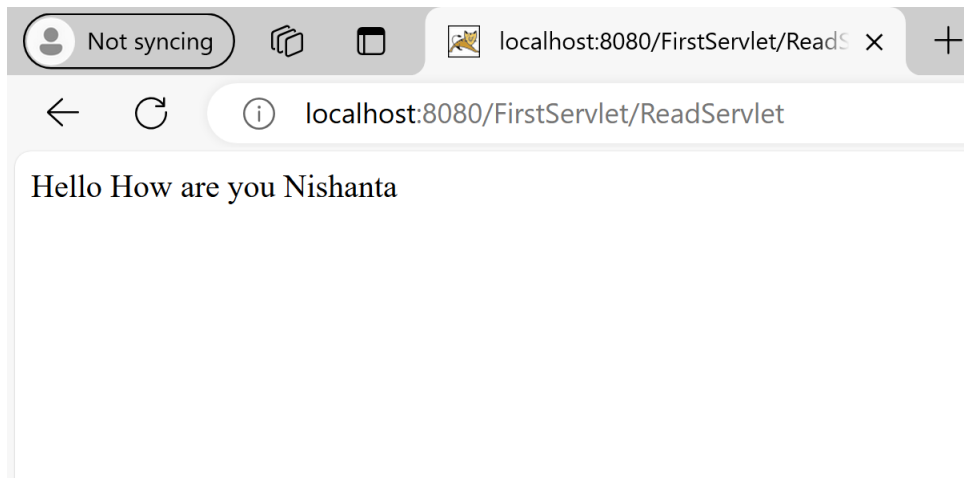
public class ReadServlet extends HttpServlet
{
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();

        Cookie[] cook = request.getCookies();

        if (cook != null)
        {
            pw.println("Hello How are you " + cook[0].getValue());
            pw.close();
        }
    }
}
```

Output





Session Tracking

HTTP is a “stateless” protocol which means each time a client retrieves a Web page, the client opens a separate connection to the web server and the server automatically does not keep any record of previous client request. Thus, each time user request to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize particular user. Session simply means a particular interval of time.

Session tracking is a way to maintain state(data) of an user. It is also known as session management in servlet. There are four techniques used in session tracking.

1. Cookies
2. Hidden Form field
3. URL rewriting
4. HttpSession

HttpSession object

Servlet provides HttpSession interface which provides a way to identify a user across more than one page request or visit to a web site and to store information about the user. The servlet container uses this interface to create a session between a HTTP client and an HTTP server. The session persists for a specified time period, across more than one connection or page request from the user. We can get **HttpSession** object by calling the public method **getSession()** of **HttpServletRequest** as below

HttpSession session = request.getSession();

You need to call **request.getSession()** before you send any document content to the client. Commonly used methods of HttpSession interface.

1. **Public String getId():** Returns a string containing the unique Identifier value.
2. **Public long getCreationTime():** Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.

3. **Public long getLstAccessedTime():** Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.
4. **Public void invalidate():** Invalidates this session then unbinds any objects bound to it.

Session.html

```
<html>
<body>
  <center>
    <form name="Form1" action="SetSessionServlet">
      Name: <input type="text" name="unname">
      <input type="submit" value="Submit">
    </form>
  </center>
</body>
</html>
```

SetSessionServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class SetSessionServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("unname");
        out.println("Welcome on board " + name);

        // get the HttpSession object
        HttpSession hs = request.getSession();
        hs.setAttribute("unname", name);

        out.println("<a href ='ReadSessionServlet'> visit Here </a>");
        out.close();
    }
}
```

ReadSessionServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ReadSessionServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();

        HttpSession session = request.getSession(false);
        String name = (String) session.getAttribute("unname");
        pw.print("Hello how are you " + name);
        pw.close();
    }
}
```

Output

